

Rubicon principles

Work in progress. This page is mirrored automatically from `rubicon/docs/principles.md` in the Causal Map repository and changes as the design does. The section written for whoever changes the code is not published here.

Rubric and Rubicon share a root, Latin *ruber*, red. A rubric was a heading written in red ochre in the margin of a law book, and the Rubicon was named for the red clay of its bed. Red means take me seriously. Crossing the Rubicon is what this app does. It's up to you to work out which river we are crossing.

What Rubicon is

Rubicon is an experimental wing of the Causal Map code base, with its own page and its own database. If it goes further it becomes a separate product. Its projects are Causal Map projects, so it can read the project's causal map, and each source's map, alongside the documents. But it has its own database and apart from sharing projects and their sources, there is not much connection between them. Rubicon writes nothing back to the Causal Map project, so deleting Rubicon tomorrow would leave the Causal Map project exactly as it was.

It is a way to implement the proposed **open format** for qualitative text processing workflows in evaluation.

Rubicon is not finished, not priced and not yet publicly available, and the working name might change.

How it differs from Causal Map. Working in Causal Map is all about creating essentially one cognitive causal model of the ideas in the texts. From that model you can extract answers to many evaluation questions; but doing so is not always straightforward. Rubicon is built to answer evaluation questions. There is no overall model of all the texts. You answer each question in its own way, using a different workflow for each, or build one workflow using the results from other workflows.

How a piece of work goes

You pick a project, state a question, and argue out a workflow with Ruby before any analysis runs: which sources to read, how to mark them up, how to count what was marked, how to combine the results, how to reach a verdict. Then it runs, and each step's result sits under the step that made it. When one result answers the question, you accept it.

Fully specified means every decision is settled before anybody sees the evidence. That is the hardcore end of a spectrum rather than a requirement.

A simple workflow

1. State the [question](#), and which part of it this workflow answers.
2. Name the sources, with a [sample step](#).
3. Write the [coding](#) instruction. Where it lets the model name what it finds, plan here how those names are put under one list before the counting rule counts anything from them, per [Consolidate what a coding named, by a rule](#).
4. Write the [counting rule](#).
5. Write the [rubric](#).
6. [Validate](#) it on synthetic texts.
7. Register it with the commissioner, dated, before the analysis runs.
8. Run it, and report what comes out.

This is the simplest kind of workflow. In practice, not every problem is even a coding problem or a counting problem. Workflows get interesting when they build upon one another, split and rejoin, etc.

Validating and registering are independent of each other: a registered workflow may be useless, and a validated one may never be registered.

Where people belong in it

The eight steps read as though one person writes a workflow and a machine runs it. That is the extreme case. Most evaluation worth doing has other people in it:

- **Before step 1**, agreeing what the question actually is, which is usually a negotiation rather than a briefing.
- **At steps 3 to 5**, when somebody who knows the setting reads the coding instruction and the rubric and says which distinctions will not survive contact with the material.
- **After the first run of a coding step**, reading what came back, fixing the instruction, running it again. This is the loop the engine was built for: run one step, look, revise, continue.
- **At substantiation or member checking**, where the people described get to say whether the description is right.
- **At sense-making**, where a group reads the results together and decides what they mean, which is not the same as being told what the machine found.
- **At the verdict**, a person may override it, and the override is retained alongside the machine's version rather than replacing it.

A workflow should say which of these it uses and where. This is not yet fully built.

What we claim, and what we do not

We claim no determinism and no full reproducibility. The engine sets each call's temperature, and no workflow can choose one. Gemini 3 models run at 1.0, the setting Google recommends for them: below it they can loop, and on 21 September 2026 gemini-3.1-flash-lite at 0 wrote the whole output limit on every call of one coding task, while gemini-3.8-flash was no more repeatable at 0 than at 1.0. Other models run at 0, except the Claude models that refuse a temperature, and Claude when asked to think, which are sent none. So AI coding varies between runs, and machine agreement with human coders is well short of perfect. Run twice over the same 19 households, one workflow's household counts per factor differed by up to three, and the consolidated lists grouped some factors differently. So read a count from one run as accurate to within a few units, and a consolidated list as accurate to within its grouping. What we do claim is **auditability**: a reader who disagrees can find out exactly where, and **reconstructability**: anyone can take the workflow document, the sources and the open format specification, and rebuild the workflow and even run it, without special software.

Projects

What you land on in the app is the project hub: every question anybody has asked of this project, what to do next on each, and a standing conversation with Ruby that is not attached to any single question and is shared by everyone on the project. Then you pick the project you want to work on.

Project notes. The hub also has the evaluator's standing notes on the whole project. Ruby is given them in every conversation on the project, whatever the question, so keep them to what matters across all questions, such as who the evaluation is for.

Making, editing and duplicating a project

A signed-in visitor can make a new Causal Map project from the project list (Home), and can duplicate the one they are on, from Project details on its hub. Both write to Causal Map's own projects, sources and links tables, under the same row-level security the app itself writes under: the write Rubicon makes is the write the app would make, not a second one that might drift from it. There is no Rubicon function or table behind any of this. A duplicate is all or nothing, made by one database function (`public.duplicate_project_atomic`) rather than the app's own several separate writes, because Rubicon should not repeat the app's own acceptance of a partial copy on failure.

Sources

The project's documents: whatever was imported into Causal Map, such as interviews, reports, minutes and field notes, and the background documents Rubicon keeps for the project. Rubicon reads their text and never changes it, because a step quotes a source by character offset: renaming a source's title, which the hub's own Sources table allows, touches nothing a quote points into. A source is never a result.

Registering a workflow records what was decided before the evidence was seen; it does not freeze the workflow. Revise as often as the work needs; each revision is a new version. See [Worth is declared by a person](#).

On the hub, Project details lets an owner or editor rename the project, edit its description, switch it public or private (private needs a paid plan, and the database says so in words if it does not have one), and choose which region its AI work runs in. A project locked read-only in Causal Map cannot be edited here either, the same rule the app applies. Collaborators can be listed and added, not removed, from here.

None of this deletes a project, or a source, or anything else. Not hidden behind a setting: absent. Deleting stays the app's, and stays something nobody does by accident from either surface.

The sources table

A plain list on the hub: each source's title, filename and size, without the custom columns or the app's own richer Sources tab. An owner or editor can add one, by pasting text or choosing a file read through Causal Map's own importer (the same route the background-document upload uses), and can rename a source's title in place. There is no delete here, and no editing of a source's text: both stay the app's. Adding a source, or changing anything about it, can leave Rubicon's meaning-search index behind the project's real sources; the next question that needs to search it reads the changed one again then, which the hub says at the moment of the edit rather than leaving it to be discovered later.

Background documents

A theory of change, a funder's standards document, terms of reference, an interview guide. They are background documents, held at project level.

They are kept in one of two places. On the project hub you upload a file or paste text, and Rubicon stores it in its own table rather than in Causal Map. Each has a title, unique on the project, which Ruby uses to open it; you can rename it in place or delete it. Its text never changes, because steps quote it by character offset. Deleting one removes it from the hub and from Ruby's list. The other place (deprecated) is a Causal Map source whose custom column marks it as background; several spellings of the column are accepted.

Both kinds are treated the same way from here on. A [sample step](#) leaves them out of its frame and lists them as left out, except where the same step goes on to narrow to passages. So a workflow that samples first never codes or counts them, while a coding step without a sample in front of it still sees them. A sample with [include_background](#) reads them like any other document, and a [where](#) on the title picks one out. A quote from one has character offsets like a quote from any source, and the record marks the document as background.

Ruby is given the list of both kinds, each by title and size, and never the text. Where a question needs one she opens it a page at a time. Reading one before a run is allowed, because it is the standard, not the evidence. She writes what she takes from it into the workflow itself: the columns, the levels and what they mean, the rubric rows.

Units

What gets counted, and what a [number](#) counts across: a person, an organisation, a document. Every threshold needs one, because "more than half of stakeholders" has to know how many there are. Rubicon makes one unit per source and has no other way of making one, so a project whose units are really people, with several interviews to a source, cannot yet say so.

Chunks and segments

A [code](#) step reads each source whole in one model call, up to a limit counted in segments, and past it in the fewest near-equal pieces that each fit, per [Reading a corpus](#). A step naming a [chunk_size](#), or a coder that cannot keep account of a whole document ([model.segment_limit_for](#) says which), cuts it instead into equal chunks of at most that many characters, sixteen thousand where it names none. Where an earlier step already narrowed a source to passages, each passage is a call.

Questions

A project usually has many questions, and they are the unit of work: you ask one, argue out a workflow for it, run that, and accept one result as the answer. Several workflows can attempt the same question from slightly different angles. Asking the same thing twice does not make a second question, because every run is filed under the question's own text.

Which question a run is filed under can be changed afterwards, from the workflow itself, since a run that turns out to bear on something else was still filed by where it was started. It moves to any other question on the project, or to one written down at the moment of moving. The run alone moves. The conversation that proposed it is a run of its own and keeps the question it was had under.

Before any of this, the whole source is split into numbered segments, and each chunk includes the markers for its own segments, so segment 12 is segment 12 in every chunk.

Segment accounting. Asked to find what matters in a long transcript, a model finds a few good things and stops. So the coder must account for every segment: either code it or refuse it, and a refusal has to quote the segment and say briefly why. [segment_quota](#) and [min_per_segment](#) set how strict this is, and the asset reports coverage. How strict to make it, a minimum of 2 on dense material, is under [Reading a corpus](#).

Passages and the index

By default a step reads whole documents. Optionally, a [sample](#) step can narrow the reading to just the passages about a subject, either by having a cheaper model screen every passage or by searching them by meaning; which to use is under [Reading a corpus](#). A meaning search (RAG) uses Rubicon's own index: each source is cut into pieces of about twelve hundred characters, each piece records the character range it came from, and each is turned into a vector and stored with the runs, outside the project. Because each piece has its character range, a quotation found in it resolves against the source, and the next step reads a window of surrounding text, not just the piece.

A step embeds the sources it will search the first time it needs them, so you can narrow on a project nobody has prepared. A source is embedded again only if its text has changed (by content hash). On a large corpus this takes minutes to hours, and the step reports how many documents are done. Each document is saved as it finishes, so a run that stops halfway leaves the finished half indexed.

A [check](#) never embeds: it reports that the step needs a run and what the run would embed. Once built, the index is reused by every later run at no cost.

Boilerplate (for RAG). A passage appearing in nearly the same words, in nearly the same place, in three or more documents. Wording counts for 0.7 and position for 0.3, scaled by length, and anything under ninety characters is ignored. Boilerplate is marked, not removed, and a meaning search leaves it out unless the workflow asks for it; otherwise a form header, faintly similar to every query, outranks the passages that answer it.

You write a new question in full, however roughly. Ruby picks it up at once: she gives it a short name for the list and for reports, says what she takes it to mean, and proposes how to answer it. That costs one turn of hers and nothing more; no workflow runs until somebody presses Run on what she proposes.

A question counts as answered when you have accepted one of its results as the answer. There is no separate "answered" switch to set: the app works it out from the accepted answers. So a question cannot show as answered without an accepted answer, and a report can list the questions that still have none. See [Reports](#).

Well-defined questions

Two careful readers, with ordinary graduate-level social science and no special training, would apply the coding instruction much the same way, and every row leads back to a quote a third person could check. Being well defined is what Ruby refuses a question for, and nothing else is.

- A property of the question and the material together, usually achieved rather than discovered. Most questions arrive ill-defined, and making them well defined is Ruby's job.
- **A vague question is welcome, and never answered as it stands.** A direct answer decides for itself what was meant and says it confidently. Say that it cannot be answered as put, and break it into parts that can. Where the breakdown turns on choices only the evaluator or their stakeholders can make, name the choices and say they may need to check back.
- Not the same as objective, factual, measurable or important. A well-defined question can be trivial, and an excellent question can be ill-defined.

Ruby

The assistant. She says what she takes a question to mean, argues with the parts that will not work, and proposes a workflow. She never decides what counts as good.

- **Ruby is the way into the app** rather than a tool beside it.
- **She is a methods coach, not an order-taker.** A question can be well defined and still a poor use of the material: only two speakers out of seventy address it, or it counts how many said something where what matters is what they said. She says what she would ask instead, then writes the workflow for what you asked. The evaluation is yours. What she drafts is a proposed workflow: she can be wrong, somebody who knows the method better should change it, and every version stays on the record.
- **What gets agreed is the workflow**, not the conversation.

She can look at what has run. Before any run she knows the corpus only by its names and columns. Once runs exist she can ask, with **look**, to see what a run did and made, one result in full, a coding's rows or a narrowing's passages a page at a time, a step's model calls and replies, or the text around a quotation. Every page states which rows it shows out of how many. She can look at any run on the project and at nothing outside it. Looking has its own limit on rounds; the turn's cost limit covers looking and drafting together. Whatever she writes next records what she looked at: where she proposes or changes a rubric after seeing results, she says so, and the evaluator still decides the thresholds.

Workflows

How a workflow is written, the file, the step kinds, the recipes and the rules that keep a finding right, is on [the drafting page](#). It is Ruby's instruction, and the reference for anybody writing a workflow by hand.

One approach to a question: a set of steps, the graph joining them, and a name. A question may have several workflows, each a different way of answering it.

- **You never number the steps.** Each step lists what it needs and what it makes, and the engine works out the order. Two lines of work can run side by side and meet later.
- **The name is the one part somebody chooses.** It identifies the approach. Ruby says how each workflow differs

- A matter of degree, and a property of the question. Checking the answer afterwards is a separate matter: an answer can be traceable and sound and still answer a question nobody asked.

What gets refused, and what does not, is under [What it refuses, and what stays soft](#).

Accepting an answer

Flagging one result as the question's answer. You accept a result, not a whole run, so a table can be accepted as readily as a judgement. A question has at most one accepted answer at a time; the database enforces this with a unique index.

Archiving

Hiding runs, or a whole question, when you are finished with them. A workflow shows as archived once every run of it is. It never deletes.

What she is given about the project. The corpus by its names, sizes and columns, never the text. The list of [background documents](#) by title and size, which she opens by name. The questions already asked and what came of them, and the registered rubrics. The [project notes](#), in every conversation on the project, kept apart from the notes on the question in hand.

Conversations

An exchange with Ruby about a question. A question can have many conversations and so can a workflow; neither owns the other. A conversation is a run like any other and has a status, but what it produces is a workflow or nothing, so the status shows whether the conversation ran, never whether the workflow is any good.

Rewriting a turn. Edit anything you said and the chat continues from there, dropping the turns below. The result is a fork: a new conversation that reuses the earlier turns of the old one and leaves out the later ones. The original is unchanged. Nothing is copied: the fork stores only the turn where it was cut.

A conversation is public where the project is. The same access rule applies to it as to any other run, so a collaborator can see it, and so can anybody at all where the project is public. The reasoning is under [The ledger, and who may write](#).

from the others. A workflow without a name is refused.

- **A workflow's state** is one of: not run, running, waiting, failed, candidate, accepted, archived. The project hub works out the next step for each question from its workflows: ask it, read what Ruby said, run the draft, watch a run, read what came out, or it is answered.

Draft workflows

A workflow Ruby has written and you have not yet run. Read it, argue with Ruby about it, change it. Checking that it will run costs nothing, and nothing is spent until you press Run.

Versions of workflows

One complete statement of what to do, under a workflow's name: the steps, the documents to be read, and the code that reads them. Change any of those and you have a new version. Versions sharing a workflow name are refinements of one approach.

- **A version is named by what it is.** Its name is six characters derived from its specification, `f350ae` rather than `v3`, so the same recipe always gets the same name and two people cannot claim one name for different work. The page also shows a number a person can say, "version 3", counting the workflow's versions in the order they first appeared, archived ones included. That number is never stored; the code is the identity.
- **Versions form a tree rather than a line.** Two refinements of one approach, such as `foo f350ae` and `foo 7d33ae`, are siblings, and neither supersedes the other.
- **The test that separates a version from a run.** If somebody changed what should be done, it is a new version. If only the running went wrong, a simple failure such as a dropped connection or a killed process, it is another run of the same version.
- **The documents and the code can change without anybody deciding anything about the workflow.** Documents are added, deleted or edited, and we fix the engine. When either changes a verdict, the page shows which one changed.

Runs

A run is one attempt at a version, with its results filled in as far as it got. It may not finish, so it also records how far it got, what it cost and whether it crashed. A version run three times is one workflow and three runs.

A run contains its results. It is the workflow as executed, with each step's outputs under it, and it refers to nothing outside itself: each number records the set it counted over and that set's size, each quote its source and character range, and each result the results it was made from. Results a step read from, or took whole from, another run are copied in, so every trail back to the documents ends inside the run.

A run records order, results, model, duration, cost, who started it, the version it ran and the engine that ran it, and every model call with the prompt as sent and the answer as returned.

Running twice is repair or a reliability check. A version's runs differ only in when each ran, how it ended and what it cost. Changing the instructions or the sample makes a new version.

Running a step at a time

Work a step at a time: run the coding, read it, fix the instruction, run it again, then go on to the counting.

- **until** stops at a named step. `continue_from` picks up what an earlier run made, recording the inherited steps as skipped. `rerun` does named steps again. On the command line, `--until <step>` when starting and `resume <run-id> --rerun <step>` afterwards.

Registering one or more workflows

Fixing the method with somebody outside the work, usually the commissioner, with a version, a date and a named person, before anyone reads the evidence. The whole workflow is registered: the sample, the coding instruction, the counting rule and the rubric alike.

Rubicon builds only the internal part of this. Registering with a commissioner or a public registry happens outside the app, and the app does not record it. Pressing Run registers the workflow and vouches for the method. The record shows who ran it and when, with no justification required. If you don't get it, don't do it.

Registration is a dial rather than a gate. A workflow may be left unregistered, registered with its changes logged, or registered with no changes allowed; the commissioner asks for the setting the stakes deserve, and Ruby names the setting she assumes. Registering the sampling and the coding commits the descriptive half and leaves the verdict open; registering the rubric too says what counts as good before the material is known. A workflow written up afterwards, describing the arrangement settled on after experimenting, is still a workflow somebody else can read, check and re-run.

Registration records ordering rather than commanding fixity. Iterating after seeing results is normal and allowed; what registration removes is changing the method without saying so. A sampling frame settled after a first look, or an instruction narrowed until a finding firmed up, moves an answer as far as a moved threshold, so each is a new version.

A registered part is versioned, never rewritten in place, and each run records the version in force. For why a declaration of worth needs that trail, see [Worth is declared by a person](#).

- `rerun` cannot name a `for_each`, whose steps are called `test_link[...]`, one per item run. Name the item run, such as `test_link[11]`.
- A run told to stop is recorded as `cancelled`. `rubicon.stop_run` writes the request into the run's row; the engine checks it before every model call, so the run stops within one call. What it finished is kept, and it can be continued.

What stops a run that nobody stopped

Three limits stop a run nobody is watching, so a mistake cannot keep spending.

- **A ceiling on model calls:** twice the estimated calls plus ten. The page, the command line and a resume all take the estimate from the same free check. The command line refuses a workflow the check refuses; a resume works out an estimate if the run had none.
- **The allowance of the account that started it,** read from `public.get_user_ai_credits_status`. If the credit check fails to answer, the run carries on.
- **Two runs at once per account.**

The first two end a run as a stop request does, so it can be continued. The third refuses the start and names the runs already going.

Budgets, `max_turns` and `max_cost_usd`, limit an agentic step and are checked where the stop request is.

Reuse and inheriting

Rubicon avoids paying twice for the same work, in two ways. **Reuse** answers a single model call from an identical earlier call. **Inheriting** skips a whole step that an earlier run already did with the same settings and the same inputs. So running a workflow again over an unchanged corpus costs nothing, and a new version pays only for the steps it changed. A draw also records its seed, so it picks the same documents again. What can still change between runs is the material (sampling reads the project as it stands) and the models' answers where the text changed or reuse is off.

Reuse answers a call from an earlier identical call. It is on unless a step sets `reuse: false`. It is keyed on the request as sent (source text included) and scoped to project and model, so an edited document gets a fresh reading. Turn it off to measure how much a coding varies between runs. Each reused answer is recorded as reused, with the answer it came from.

Inheriting a step skips the whole step. One rule decides it (`Ledger.reusable`): a step is taken from an earlier run on the same project that finished the same step (by spec hash) over inputs with the same contents, made every output this step declares, and, for a judge, applied the rubric's current version. An input's contents are its quotation rows (source, unit, text, offsets, how each was located, the values coded on it) plus its payload, minus the call bookkeeping `calls_fresh`, `calls_reused` and `answered_by`, which a judge or note reading the coding does not see. A pointer to the screen of passages that cut against the reading is compared by what that screen contains, since a sample gets a new id every run.

`hashing.asset_hash` defines this; older codings hashed over their whole payload are compared by the same rule. The engine looks first in the run being continued, then in the last run of the same workflow, then in the rest of the project under any workflow. What is taken is copied in with its quotes and whatever it cites, each row naming its source asset. A new version redoes only the steps it changed; any difference, even a reworded instruction or a different chunk size, makes a different step. `rerun: [step]` and `reuse: false` force the work to be done, and a step named in `rerun` also bypasses the call cache. Where a step is not taken, the check reports whichever test the closest earlier step failed (`Ledger.reuse_or_why`): it is new, the evaluator changed it, its rubric was revised, or its input changed because an earlier step is being redone. A sample is never taken; it is compared with the last draw, which is the same, or differs because the documents changed or the step's code was revised.

Results

What a step produces: a sample, a coding, a table, a number, a judgement, a paragraph. Its type is one of the seven the engine knows, listed under [The words the code uses](#). For example, in the public contribution analysis of the proposed UK research funding agency, a **sample** step produced the nine documents it drew (a `source_set`), a **code** step produced 307 coded passages (`quotes`), a **compute** step produced a table of counts per link of the theory of change, a **judge** step produced a verdict of partially supported, and a **note** step produced the annotated theory of change.

- A source is never a result. A sampling step produces the sample it drew; the documents remain Causal Map's.
- **A result never changes.** Running the step again adds a new result beside the old one.
- **Every result records which results it came from.** That chain is its lineage.

For example, edit only the rubric in the running example and run again: frame runs (a sample always does), `code_borrowing` and `count` are taken from the earlier run, and `verdict` is done again because its rubric was revised.

The page shows what a change does below it. An inherited step is recorded as `skipped`, naming its source run; the page shows it as reused. For a draft, the page shows each step as taken, done again (nothing earlier matches, or it reads a step that will be redone), or drawn again. The preview comes from the check and matches the run, with one exception: the check reads no cached answers, so a step whose calls are all cached can let the run take later steps the preview priced as done again. The page refreshes the preview whenever a draft is opened. A step made by an older revision of its step type's code is marked stale, and is the only step the page offers to redo. A run shows which of its steps a later run of the same workflow redid. Where a newer run of the same version under the same question has finished, the accepted answer and the report's footer link to it by date and the start of its id; this notice is never called stale. A newer run does not replace the accepted one: the report states once which run the accepted finding came from, and whether it was accepted before the newer run existed, in preference to it, or with the same result. A saved copy links to the app's public address, `APP_ORIGIN` in `webapp/env-config.js`.

A step that reads the project is never inherited. A **sample** step always runs, taking no model calls and under a second. A coding step given no sample reads every document, so it is never inherited either; where nothing changed, its calls come back from the cache.

A sample fingerprints what each document said, with one short hash per document, so an edit changes the sample's hash and nothing downstream inherits. A finished run is marked stale when a sample it drew includes a document since edited or removed, or, for a whole-project sample, when documents have been added. The warning appears on the run, on the accepted answer based on it, and in the report's footer.

The engine's commit is recorded on the run and gates nothing. Inheriting is blocked only where it would give a wrong answer: an edited document does; a change to our own code does not.

A version's name is derived in one place and stored wherever a specification is. It is six hex characters of a hash of the workflow, excluding the declared `version` number and the project name, and ignoring whitespace at the ends of strings. The engine stamps it on the run and the draft; the page hashes nothing. Older runs show their declared number instead, as `v3`.

Result set. Everything a workflow's runs produced, finished or not. A run that failed at its fourth step still produced three results, and they stay in the set.

Lineage

The chain of results each result came from, ending in quotes and the words of a source. If the chain breaks anywhere, the run counts for nothing.

You can walk back from any claim: a result lists the results it came from, a number opens the rows behind it, and clicking a quote opens its source with the words highlighted. In the contribution analysis, the verdict that the first link is well supported cites six passages; the verdict came from the table of counts per link, the table from the coded passages, and each passage from a position in a source, such as characters 3783 to 3978 of the oral evidence of 23 September 2020.

Quotes

A marked passage: the words and their position, checked against the source text. For example: source oral-2020-09-23, characters 3783 to 3978, match **exact**, text "On the technical management side of DARPA, which is about 100 people at any given time, we all do limited-term tours. We know that when we show up; our dates are on our badges, and then we leave." The passage also has the coding's columns, here a stance of supports and a certainty of evidenced. A quote the engine cannot find in the source is dropped: a plausible quotation that appears nowhere is the most dangerous thing an AI can produce.

Match tier. How a quote was found. **exact**, **canonical** and **gapped** snap the quote to the source's own words with exact offsets; **fuzzy** retains the model's wording with approximate offsets, labelled as such. A quote that cannot be found at all is dropped and counted, never stored.

Numbers

One value a **compute** step produced: a single number, or a table of numbers where it grouped. The workflow file and the code call these figures (a **compute** step lists them under **figures**), so "figure" and "number" mean the same thing here. Each states the unit it counted across, the filter it applied, what it took from each unit and how many units it considered. Its name must be unique in the workflow; reserved names are refused. For example, a number named evidenced_support, written `{reduce: count, over: {group: proposition}, where: {stance: supports, bearing: {at_least: moderate}, certainty: {at_least: reported}}}`, counts the passages under each link of a theory of change that support it with at least moderate bearing and at least reported certainty: 55 under the first link, 25 under the third, none under one rival explanation, and 8 of the 9 propositions (six links and three rival explanations) with at least one.

A number that compares groups comes with a test of whether the difference could be chance, in small print under it on the run page and in every copy of the report. A count of the households that keep livestock, grouped by the sex of the household's head, might read "Fisher's exact test on the share of each group that matched, $p = 0.37$, $n = 19$: a difference this size could easily be chance." Over coded passages it reads like "Passages reshuffled by document: $p = 0.04$, 19 documents, 63 passages. A difference this size is unlikely to be chance."

Reports

A report gathers the accepted answers of several questions into one document, a section each, and also lists the questions that have none. Each section shows one question's accepted answer. Where that answer is a **note**, its findings cite passages, and an annex prints them.

- Two shapes compare groups: a count grouped by two columns (a cross-tab), and a count or share grouped by one column with a **where**.
- Tables of separate cases, such as documents counted by their own columns: a 2x2 table gets Fisher's exact test; a larger one gets chi-square, or Fisher's where some cell would expect fewer than five. The small print warns when cells are small, and reads too few to test where no arrangement of the numbers could show a difference.
- Coded passages, several to a document: chi-square on the passage table, with p the two-sided share of reshuffles giving one at least as large, where each reshuffle moves whole documents between the groups. Every distinct reshuffle is tried where there are at most 200,000, otherwise 10,000 seeded random ones, so the result repeats exactly. One side of the table must be a property of the document, such as the sex of a household's head; the engine finds which from the data. Where both sides vary within documents, the small print states that it was not tested.
- A group with a single document is too few to test. A breakdown with no **where**, a mean or a sum per group, a share across a cross-tab and a ratio get no test.

The arithmetic is in `scripts/causalmap_report/stats.py`, shared with Causal Map's own reports.

Judgements

One verdict from a **judge** step: what the material earned, why, and which quotes support it. Some verdicts are settled by arithmetic over numbers, others by the AI reading the evidence against the written description; the judgement records which. For example, a **judge** step applying a rubric to the six links of a theory of change returns one verdict per criterion (the first link, a gap in the existing funding system, well supported, citing six passages; the budget link contested, citing three) and an overall verdict of partially supported, each with its rationale.

- **A criterion about the method cannot cite the material, so it is answered without quotations:** whether enough interviews yielded anything, whether one category took nearly everything, whether a comparison has a big enough base. Leave to the reading model only the criteria that need the words, such as whether the passages returned belong in their category.

Evaluative judgement. A judgement that restricts a set of options defined and ranked by worth. Registration most often fixes one in advance, though a commissioner may register any part of the method. The three conditions are under **Worth is declared by a person**.

The editor's layer

A report's findings are the accepted results, unchanged. Around them a report may include an editor's text, which the editor model writes from the findings, the annex's method text, and the project's document counts, split by any column that sorts them into a few groups: a summary, a lead-in of one to three sentences before each finding, a plain account of how each was made, and a glossary of the terms the report uses. It may add nothing a finding does not contain, and must stay at that finding's scope. If a finding says 8 of the 12 households that mentioned water had a new well, the summary may say that, but not "two thirds of households had a new well" when the study interviewed 21.

- Code refuses the text unless every number in it, in digits or in words, appears in the findings it names (or, in the summary, in the project numbers), it cites no passage and quotes nothing, each summary point names its finding, and each glossary term is one the report uses.
- A cheaper model, the editor's scope check, then returns any sentence that goes beyond the finding it names: a wider scope, a count or fact the finding does not give, or more certainty than it has.
- Either check sends its problems back for one more attempt. A second refusal saves the report without the layer, with the reason, unless the only fault left is a glossary entry for a term the report does not use: that entry is dropped and named in the record under [glossary_dropped](#).
- The layer is marked as the editor's wherever it appears and is not an accepted finding. It is recorded as a run named [editorial](#) with one step, metered like any other; the page shows the run id, the model, the cost and the check's result with the report.

The workflow canvas

Ruby's conversation on one side and the work on the other. The work sits under a trail of levels, each inside the one before it: everything, a project, a question, a workflow. Each level is an icon that opens that level's page, and beside it the one chosen there, a dropdown of its siblings in the way the project box is. Only the levels there is something to show under are drawn. The level you are on is marked, and going up a level keeps the ones below it, so Everything and back does not lose the question.

- **The project page lists its questions on the right, and the question page lists its workflows on the right.** Each list is the way into what is inside the level.
- **A workflow is one scrolling document of steps and nodes, and it holds no detail.** A node is anything a step read or made: a set of passages, a coding, a table, a verdict. The canvas names each step and each node and does nothing more; before a run it shows the steps, and after one each node stands under the step that made it.

Principles

The rules that apply to writing a workflow, the recipes and the named methods among them, are on [the drafting page](#). What follows applies to everything Rubicon does.

Repeated openings

Each note opens with the finding that most changes how its question should be read, so several sections of one report may open with the same passage (the same stretch of the same document, whatever its quotation id). Only the first prints it. Each later finding gets one sentence naming the finding that opens with it, and its own opening paragraph moves, word for word, into its part of the annex. It is never rephrased, since a paraphrase would be a claim nobody accepted. None of this applies where the editor's text is present.

Passages in the annex

A report prints every passage a finding cites, found by the quotation ids in the finding's handles (a finding can cite a passage in its prose that is missing from its evidence list). A handle whose passage cannot be read is listed, not dropped. Each note numbers its handles from q1, so in a report each handle is prefixed with its section number (2.q5); the ledger keeps the original. Under each passage the report lists its coded values, including its theme, and its document's value for every attribute a number splits or filters by, so a reader can check a group comparison passage by passage.

Bundle

A report can also be saved as a zip for somebody who has to check the work. It contains the report and, for each accepted answer, the run behind it in the open format with the full text of every document that run read, so each quotation can be checked using only the files. A README explains the folders, the format version and where it is published, and whether the checker accepted each record. Because it contains whole documents, give it only to somebody entitled to see them. It does not yet include [background documents](#) kept on the project hub.

- **Click a step or a node, in the canvas or on the map, and it opens in a lightbox.** A step opens what it was told to do, how it went and what it cost; a node opens what is in it, such as a coding's quotes or a verdict and its rubric. Clicking a quote opens the source around it.
- **A margin map is fixed beside the document,** one viewport tall: its nodes are the steps and results, its edges the parentage. Click a node to open it and scroll to its row; the row you are reading lights up. For the contribution analysis the map is a chain of five steps, from the sample through the coding, the counting and the verdict to the annotated theory of change, each with its result beside it.
- **A step may depend on several earlier results or sources, and a result always has just one parent step.**
- **Every run names the models its steps actually read with,** wherever the run is named: the head of its page, its workflow's card and the list of that workflow's runs. They come from each step's own record, not from what the workflow asked for, and the cheaper readers that Draft and Test put a step on are marked.

Worth is declared by a person

Rubicon never decides what counts as good. A person sets the standard, in advance and under their name; the AI applies it to the material and shows its working.

Julian King put the objection in October 2025: AI should not be making evaluative judgements about human affairs. The evidence-synthesis bodies draw the line in the same place, where a model makes or suggests a judgement (Fleming & {Noel-Storr 2025}), and the case that the technology can strengthen qualitative inquiry is being made alongside it (Wise et al. 2026). We agree.

- Looking at the material cannot tell you where adequate begins, and a standard set after seeing the numbers can be made to fit any of them. So the standard comes first, from somebody who put their name to it.
- **Offer the form of a standard, never its numbers.** Ruby does not supply a threshold of her own, such as "overwhelming means eight of ten": she shows where a threshold goes and asks the evaluator for it.
- A registered workflow is versioned and never replaced, and records who registered it and when (on the run path the machine writes this). Freezing a version, and recording whether the freeze came before the run, is designed but not built.

A step is making an evaluative judgement when all three of these are true. They can apply to a coding column ("code the passages where the partnership was meaningful") or a counting rule (a threshold of more than half) as well as a rubric. The app does not test them: it treats a step of type `judge` as the evaluative one, asks for a standard in its rubric, and does not remark on a value declared anywhere else.

- It **restricts** a set of options declared in advance, usually to one of them, sometimes to a band such as "adequate or better".
- Each option **states what it takes to earn it**, in enough detail and context that somebody could hold it against real material and say whether the material meets it; a description that only gestures leaves the reasoning to be rebuilt case by case. "Green: every partner described a change they could date and name a cause for" is a standard; "Green" alone is not. The engine refuses a coding column whose levels are bare names. A rubric that lists its verdicts must state what each stands for; one that lists none runs with the meanings reported as missing.
- The options are **graded by worth**. Health, farming and income is a classification. A frequency table over speculative, reported and evidenced is graded but chooses no point on the distribution.

`at_least` works on an ordinal and never on a category. The test applies to the answer a step gives, so a workflow is usually descriptive in the middle and evaluative at the end; structurally, that means whether it ends in a judge step.

Transparency is not enough on its own. There has to be a place to disagree: override the verdict, rewrite the rationale, reject a cited quote, mark a criterion unanswerable. The override would be shown beside the machine's verdict, record who and when, and be what everything downstream uses. None of this is built: the only human decision the app records is accepting or withdrawing a whole result.

Every claim points at a quote

- Any narrative or verdict about the material must lead back to its quotes (Moravcsik 2014). The engine refuses to store a note or a judgement that cites none, by requiring the coding to be a declared input of the step making the claim. A table is not held to this.
- This is the likeliest way a tool like this fails, and the failure is invisible: two steps from the source the prose still reads well and the numbers still look precise, with nothing connecting

either to a document. Note it's easy for an AI tool to reach a conclusion and then search the text for quotes to back that up. Rubicon does not fall into that trap.

- **Some findings are about the method, not the material:** whether contrary evidence was looked for, whether the sample supports the claim. These cite the run record, and the judgement records which kind each verdict was.
- A criterion should be one kind or the other; the engine does not refuse one that is both. "Was anything found that cuts against the hypothesis, and how was it handled?" is two questions.
- A band should stand for one state of affairs; the engine does not read a band's description. If the worst band covers both "contrary evidence was found and it is damaging" and "nobody looked", the reader cannot tell which happened.

A negative finding is the hardest case, and the test is what the claim CITED, not what it was handed. A note that cites nothing, such as which of a theory's links the material never mentioned, is backed by the search: what was looked for, over what frame, how much was read, and how many rows it found. This is in the asset's payload as `search_record`, beside the note in the export.

Give every proportion its base, and include in the base the documents that gave nothing. A denominator built from coded passages drops the silent documents, so "of the sources that gave me something" reads as "of the sources" and a sparse finding, such as unintended harm, looks bigger than it is.

For a presence question, code the presence. Where a question asks whether something is there, such as whether households keep livestock, code only the passages where it is; a document without a coded row did not mention it. To count the documents that did, give the number a `where` listing the values of a column every coded row fills. Without one, a count over units includes the silent documents; the engine warns on such a number, under "Working with numbers". A level meaning "not mentioned" does not have a passage to quote, so its rows cite unrelated text, and every document gets a row. Keep a third level such as "unclear" only where "the document addresses this and is unclear" is itself a finding.

Beside what it found, the engine reports the passages dropped because their quote could not be located, how much of each document was examined, and how many model calls it made and what they cost.

Build the thing, and not only the verdict

A verdict compresses a whole analysis into one word, and tracing back does not rebuild it. Where a reader would expect a graph, a table of outcomes or a cross-tab, the workflow builds it and declares it as a result; it is usually a step on the way anyway. For a question about whether a theory of change held, that means the table of supporting and contradicting passages per link, beside the verdict. A table can be accepted as the answer as readily as a verdict. A question that wants a number should not pay for a graph nobody asked for.

Where the evaluator does not want criteria settled before the reading, because the material may be richer than the question anticipated, the workflow stops at the table or the prose and reaches no verdict. A free summary ("summarise the findings") is a step away from well defined, since another reader would write another one; where a summary is wanted, specify it narrowly, such as the main point of each finding.

Say what you are comparing against

A programme did well: compared with what? There are seven answers, and they are not interchangeable.

- **A denominator.** Eleven of the fourteen partners. Needs a unit and a stated base.
- **An alternative.** Which of two readings the material better supports.
- **A written standard.** A rubric agreed in advance. The only one that works when there is nothing to compare with.
- **A comparison case.** The programme before it started, a similar one elsewhere, a site that got no support. Depends on the cases being alike in the ways that matter.
- **What was expected.** The theory of change, the target, the plan. Measures delivery, not worth.
- **What would have happened anyway.** Usually the hardest; approachable through causal mapping and other generative accounts of causation. See the method page [generative-and-counterfactual](#) for when each framing fits.
- **The evidence itself.** Sometimes the finding is that nothing here is well enough evidenced for any of the above.

Counting how many people said something does not settle whether it was any good.

Search both sides equally. Iterating on what you have read can fit the method to one corpus: an instruction narrowed until it finds what you saw, a threshold set where the numbers fell. Searching both sides in one pass over the same text is a convention the app does not enforce. Where two codings are compared, it checks they had the same opportunity, per [align](#), and reports any disparity without refusing.

Hold material back. Develop the workflow on a subset, or on analogue material from a previous year or a similar programme, then run the settled workflow over the rest. Where the workflow writes its own tests, split the corpus so that the half that wrote a test never scores it. Before the real material, a workflow can be run over invented documents written to score well and badly (page synthesise-step); if it does not separate them, the method is at fault rather than the material.

Say what you did not read

A workflow may read part of the corpus, by drawing a sample of documents or by narrowing to [the passages about a subject](#); it must not report the result as though everything had been read. A number's frame is whatever reached the step that produced it.

Every result records its frame. The sample step records how many documents it read out of how many, what share of their text it passed on, and what chose that share (a screen, a meaning search, a pattern, a budget). The runner stamps this on every result built on it, through the lineage, and the page and the report print it above the numbers, the verdict and the note: a count from part of the text is a floor for the corpus, and a share or a comparison is over what was read. A result whose inputs were read differently lists each frame. In the open format the frame is each result's **coverage**, and a run's budget and the share of text read within it are on its **run**.

What a step loses is part of the frame too. A chunk whose reply could not be read, a row that failed the declared columns, a row a soft recode could not place because its label went under two values: the step adds to its frame how many passages of how many, why, and, where known, which values they would have counted towards. Later results inherit this. A count of such a value is then a floor, and every step that writes about numbers is told so.

What a reading is shown is part of the frame too. A **judge** or **note** is shown every quote in its inputs, up to a thousand unless its **quotes_shown** sets another limit. Beyond that it is shown a spread, one quote from every unit before a second from any, and told how many of how many it is seeing; the result records both numbers.

State a number inside its frames, innermost first: 28 of the 718 achievements coded mention the embassy, those 718 came from 23% of the text of 142 documents, and a meaning search chose that 23%, so 28 is a floor for the corpus. Whenever the reading was partial, every step that writes about numbers, a judge's rationale and a note alike, is given the frame and this form, and Ruby states results the same way in conversation.

[Reading a corpus](#) covers when narrowing is right and when it is wrong.

After narrowing, state an absence as an absence in the search: "No passage above 0.45 on childcare, in any of the thirty-seven interviews searched", never "Nobody mentioned childcare". The asset records what was looked for, how close a match was required, and which documents contributed nothing. The method page [reading-enough](#) covers how much has to be read before an absence or a proportion can be relied on, and how a search misses a link that is in the text.

What it refuses, and what stays soft

- **"Read the documents and tell me whether the project was effective."** In one pass, that gives a confident verdict with no stated standard and nothing specific to disagree with.
- **"Summarise these documents" and "what are the main themes"** are the same fault at a smaller size: a global judgement with no procedure. They get a workflow, a coding in the respondents' words and a consolidation, never a direct answer.
- **"Is this a good project?"** is four questions: good for whom, compared with what, against whose standard, and how much of that this material can answer. Where a terms of reference uses the generic criteria (coherence, efficiency, sustainability, relevance, equity), the method page [generic-evaluation](#) turns each into a workflow with its own standard.
- The test is whether the question is well defined. A question is never refused for being about tone, feelings or anything else that sounds soft.
- **Refusing is rarer than it sounds.** "What percentage of the budget went on training?" is answerable where a report states the number. The usual answer is a workflow that goes and looks; if nobody wrote the number down, that is the finding, backed by the search.
- **Some questions are well defined in parts and not as a whole.** "What share of the fellows who wanted a policy meeting got one?" is two codings and a ratio. A single pass would be arithmetic done in a model's head.
- **A question whose answer is in a spreadsheet is refused.** Hours between landfall and the first cash payment, or cost per person reached, come from a payments system without any words behind them, and every number here must be traceable to words somebody said. The workflow states which part of the question it answers, and you bring the other numbers in when you write.
- **A number spoken in a document is in scope.** "We reached about four hundred children", said by a school director, has a speaker and a position in a source. A coding step declares a numeric column, and the counting does arithmetic on it.
- **A judgement may refuse the question.** Where the material shows the question assumes something that did not happen, or compares things the sources cannot compare

fairly, the judge step reports this instead of awarding a band, and cites the evidence.

Decline the statistics Rubicon does not do

Ruby's stance. Rubicon does not compute effect sizes, differences in differences, power calculations, regressions, or any number for what would have happened without the programme; a version improvised from counts would be wrong in a way no reader could see. So she declines, plainly and without apology, names the kind of analysis needed, says it has to be done from the data by somebody with a statistics package, and offers the part Rubicon can answer, usually what people said about the change and why. She does not name a product or a website, or write a weaker version to have something to show.

The house phrase, to adapt rather than recite: "Answering that properly needs [a difference-in-differences estimate, an effect size, a power calculation], worked out from the data by somebody with a statistics package. Rubicon reads and counts what people said, so what it would give you is a number that looks like an answer without being one. What it can do is [the part it can answer]."

Where the refusal depends on a fact about the material, such as there being no comparison group, she says where that fact comes from, since she has read none of the documents: a column the sources have, or the absence of any column marking who was reached. She never writes "Every household was reached by the programme" into a frame description, where it would enter the record as a finding. The form is "nothing in the sources' columns marks a household the programme did not reach, so the material offers no comparison group". A coding step that reads for it is the way to find out.

Open questions

An item leaves this list once decided: built, it moves into the principles above; abandoned, it is deleted.

- **Validating a workflow on synthetic contrasting texts.** The `synthesise` step is built. Missing: a final step, run as the ordinary workflow over what `synthesise` produced, that compares the verdicts awarded with those expected and reports the separation as a number. Design in `rubicon/validate a workflow on synthetic contrasting texts xkTODO xpSTEVE.md`.
- **Near-miss reuse.** Identical requests are reused by hash. Reusing a near-identical theory of change would need the difference stated in terms a person can judge (which links, which clause) and the evaluator's agreement.
- **Reading an accepted answer rather than a raw asset.** `from_run` reads a raw asset, so a comparison can be built on a run nobody accepted without showing it. Proposed: an input `{answer_to: <question id>}` that reads the accepted answer and is refused where there is none. Design in `rubicon/xkTODO a comparison should read an accepted answer xpCLAUDE.md`.
- **Building a theory of change, rather than checking one.** Checking a link against the material is close to what the app does now. Building one, with categories that emerge from hundreds of fragments, needs an auditable design.
- **Running a step at a time from the page.** The command line has `until`, `continue_from` and `rerun`; the page runs only whole workflows. The status word for a part-run workflow must differ from `partial`, which `runs.status` uses for a run whose steps failed.
- **Whether the sample belongs to the workflow or to the run.** Running over twelve sources and then all fifty-three makes two versions, because the sample decides what the answer should be. Unsettled: what the pane shows, which needs a run picker either way.

Where the question is about a comparison, she proposes a number whose groups are a property of the documents, such as who was interviewed, since that can be tested, per [Numbers](#). A difference the test says could easily be chance is not established, however it looks in the table: say so, or leave it out. Nor does it show the groups are alike, since with small samples most real differences would not show either.

Where this sits in the literature

- Scriven's logic of evaluation separates description, "What's so?", from valuing, "So what?", then reaches a verdict through criteria of merit, standards of merit and a synthesis into overall worth (Scriven 1980, 1991; Fournier 1995).
- Davidson: an evaluation must ask not only "What were the results?" but "How good were the results?" Her worked failure is an executive summary of counts and percentages with no verdict in it (Davidson 2014).
- Gargani and King's second principle: "an evaluator cannot ascertain the value of an impact solely by studying it because value is not located within it". The philosophical question is open; Putnam's argument against the fact and value dichotomy is the standing objection.

- **Forking a workflow from a step**, discarding everything downstream, to make a new workflow. Not built.
- **A place to disagree.** Overriding a verdict, rewriting a rationale or rejecting a cited quote, kept beside the machine's version and passed downstream. The principle is above; the surface is not built.
- **A composite as a result in its own right.** `composes` builds one inside the coding's asset, so it cannot be accepted as an answer or drawn. Open: whether it becomes its own type.
- **Comparison against what was expected.** A workflow can compare two codings. It cannot yet check a written expectation against the material and report which links nobody mentioned.
- **The absent-link register.** A proposition the material never mentions shows only as a nought in a table.
- **Iterating a step until a number passes a threshold.** A bounded loop over one step, so the coverage dial settles itself. Deferred until registration defines what a version means when the number of turns is decided at run time.
- **The draft-to-run link.** A run stores its workflow but not the id of its draft, so two drafts of one recipe cannot be told apart. One column, written when the page posts the draft's id, would fix this.
- **Arranging the canvas by asking**, such as "put the two codings side by side". Ruby cannot change the canvas. Not built.

Not settled at all:

- Making the unit a person or an organisation rather than a document, when who they are is only discovered during the coding.

- Expressing a base that is not a set of documents, such as "the forty partner organisations", when only twelve appear in the material.
 - Pausing a workflow for people to do something in the world, then picking up where it left off.
- Merging results that describe the same thing, keeping the trail back to every contributing passage.
 - Naming.

References

Flemyng, & {Noel-Storr (2025). *Position Statement on Artificial Intelligence (AI) Use in Evidence Synthesis across Cochrane, the Campbell Collaboration, JBI and the Collaboration for Environmental Evidence 2025*. Cochrane Database of Systematic Reviews.
<https://doi.org/10.1002/14651858.ED000178>.

Moravcsik (2014). *Transparency: The Revolution in Qualitative Research*. APSCC Newsl., 47, 48--53.
<https://doi.org/10.1017/S1049096513001789>.

Wise, Gresalfi, & {Spencer-Smith (2026). *Why AI Is Not the Enemy: Opportunities to Strengthen Core Commitments of Qualitative Inquiry Through Trustworthy AI-in-the-Loop Analysis*. International Journal of Qualitative Methods, 25, 16094069261435579.
<https://doi.org/10.1177/16094069261435579>.